# Class 6: R Functions

Anisa Mody (PID: A19145291)

## Table of contents



### Background

All functions in R have at least 3 things

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

### A First Function

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can either be "required" or "optional". The later have fall-back default values that we be used if the user does not specify them.

```
add <- function(x, y = 1000) {
  x + y
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 1010
```

## Start of Lab 6

### Section 1) Write your first R function: add()

Question 1a. Your first version of the function should add two input numbers together. For example, `add(4,7)` should return "11".

Explanation: To define add as a function, we use "x" and "y" as inputs because the function needs values to add.

```
add <- function(x, y) {
  x + y
}
```

```
add(4,7)
```

```
[1] 11
```

Question 1b. For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add( c(4,7,10) ).

Explanation: This function is parallel to the one used in Question 1a. However, we assign "y = 0" to give "y" a default value, so the function still works if only "x" is provided. We call `add()` with a vector to test that the function works on multiple numbers.

```
add <- function(x, y = 0) {
  sum(x,y)
}
```

```
add( c(4,7,10) )
```

```
[1] 21
```

Question 1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return 2. Hint: explore the … (dots) argument or the base R function sum().

Explanation: We redefine `add()` using … because this lets the function accept any number of inputs, making it more flexible than just "x" and "y". `sum(...)` is used because it is the built-in R function designed specifically to add all numeric values passed into the function. The updated function is tested with another vector, including a negative number, to confirm it correctly adds all values together.

```
add <- function(...) {
  sum(...)
}
```

```
add(c(1, 2, 3, -4))
```

```
[1] 2
```

We can explicitly set a *return* value output from a function (rather than the default last line) by using the `return()` function call.

**Section 2) Write a generate_dna() function**

A useful function here is the "base R" `sample()` function:

```
sample(1:5, size = 3)
```

```
[1] 3 5 4
```

```
sample(1:5, size = 60, replace = TRUE)
```

```
 [1] 5 4 3 1 5 2 5 5 2 3 5 4 1 3 3 4 5 1 3 5 3 5 5 2 2 3 5 5 3 4 2 3 2 3 5 5 3 3
[39] 1 5 4 1 2 5 1 2 3 3 3 5 2 4 2 4 5 5 1 5 4 4
```

We can use this to make a random nucleotide sequence if we draw from "A", "C", "G" and "T"…

```
sample(x = c("A", "C", "T", "G"), size = 10, replace = TRUE)
```

```
 [1] "C" "T" "A" "T" "G" "T" "A" "C" "C" "G"
```

Question 2a. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return “A”, “T”, “T”, “G”, “A”, “C”.

Explanation: `generate_dna()` is defined so we can easily create random DNA sequences without rewriting the same sampling code each time. Then, `sample()` is used because it randomly selects elements from a set of choices, which is exactly what is needed to build a random DNA sequence. `x = c("A", "C", "T", "G")` specifies the four DNA bases to sample from because these are the only nucleotide letters in a DNA sequence. `size = len` makes the output sequence the length requested by the user. `replace = TRUE` allows each base to be chosen more than once because real DNA sequences can contain repeated nucleotides.

```
generate_dna <- function(len) {
  sample(x = c("A", "C", "T", "G"), size = len, replace = TRUE)
}
```

```
generate_dna(len = 10)
```

```
 [1] "T" "T" "A" "G" "T" "G" "A" "C" "A" "G"
```

Question 2b. Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a singe vector of multiple letters). For example “AAGGCTG”.

Explanation: The first few lines of this code are the same as the last question, aside from new additions. The sampled bases are stored in `ans` first so the function can adjust the output before returning it, which makes it more flexible than returning `sample()` directly. The `if(single.element)` statement is used to control the format of the result, letting the user choose between getting a vector of individual bases or one combined DNA sequence. Inside that condition, `paste(..., collapse = "")` is used to join the sampled bases into a single string with no spaces in between. Finally, `return(ans)` explicitly returns the finished result, whether that is a vector of bases or one continuous DNA sequence. The calls `generate_dna()` and `generate_dna(single.element = TRUE)` are then used to test that the function works with its default settings and that the single-string output option behaves as expected.

Note: Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`.

```
generate_dna <- function(len=10, single.element = TRUE) {
  ans <- sample(x = c("A", "C", "T", "G"), size = len, replace = TRUE)
  # cat ("hello")
  if(single.element){
    # cat ("bye")
    ans <- paste(ans, collapse = "")
  }
```

```
  return(ans)
}
```

```
generate_dna()
```

```
[1] "TGGAATGTCG"
```

```
generate_dna(single.element = TRUE)
```

```
[1] "GTGGCTATCT"
```

Question 2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example: >len9 CGAAGGCTG

Explanation: `cat("hello \\n there")` is being used to print text with a line break, where `\\n` tells R to start a new line. In the `generate_dna()` function, `cat()` is then used again to format the output like a FASTA record by printing an identifier line first, such as ">len 10 someid", followed by the DNA sequence on the next line. This is useful because FASTA is a standard format for representing sequence data, so the function is not only generating random DNA but also displaying it in a biologically meaningful way. The line `x <- generate_dna(10)` runs the function with a sequence length of 10 and stores the returned DNA sequence in "x", while `cat()` separately prints the formatted FASTA-style output to the console.

```
cat("hello \n there")
```

```
hello
 there
```

```
generate_dna <- function(len=10, single.element = TRUE) {
  ans <- sample(x = c("A", "C", "T", "G"), size = len, replace = TRUE)
  # cat ("hello")
  if(single.element){
    # cat ("bye")
    ans <- paste(ans, collapse = "")
  }

  ## Format as FASTA with an ID line
  cat(paste(">len", len, "someid\n"))
  cat(ans)
  cat("\n")
```

```
  ##

  return(ans)
}
```

```
x <- generate_dna(10)
```

```
>len 10 someid
GGTACGAAGT
```

### Section 3) Write a generate_protein() function

Question 3. Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG".

Explanation: This function follows the same functions as the DNA version, but it samples from the 20 standard one-letter amino acid codes instead of the four DNA bases so that it generates a random protein sequence. The sampled amino acids are first stored in "ans" so the function can optionally reformat them before returning the final result. The `if(single.element)` condition is used to let the user choose whether the output should remain as separate amino acid characters or be combined into one continuous protein string with `paste(..., collapse = "")`. The `cat()` lines are included to print the result in FASTA format, with a header line showing the sequence length and ID followed by the protein sequence itself, which makes the output look like a standard biological sequence record. Finally, `return(ans)` returns the generated sequence so it can also be saved and used later in the R session.

```
generate_protein <- function(len=10, single.element = TRUE) {
  ans <- sample(x = c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F",
  # cat ("hello")
  if(single.element){
    # cat ("bye")
    ans <- paste(ans, collapse = "")
  }

  ## Format as FASTA with an ID line
  cat(paste(">len", len, "someid\n"))
  cat(ans)
  cat("\n")
  ##

  return(ans)
}
```

```r
x <- generate_protein(10)
```

```
>len 10 someid
TDAAASRVAK
```

### Section 4) Generate random protein sequences of length 6 to 13

Question 4. Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand.

Explanation: This code uses the same base function structure from before, but extends it by adding a for loop to automatically generate multiple random protein sequences instead of just one. The new part is `for(i in 6:13)`, which steps through sequence lengths 6 to 13 so the function can be run repeatedly with different lengths. The `cat()` function was also added outside the function to print each generated sequence with a FASTA-like header such as ">id.6", making the output easier to read and better formatted for sequence-style display.

Notes: The function `cat()` combined with `paste()` and the newline character `\\n` makes this relatively straightforward.

```r
generate_protein <- function(len=10, single.element = TRUE) {
  ans <- sample(
    x = c("A","R","N","D","C","E","Q","G","H","I","L","K","M","F","P","S","T","W","Y","V"),
    size = len,
    replace = TRUE
  )

  if(single.element){
    ans <- paste(ans, collapse = "")
  }

  return(ans)
}

for(i in 6:13) {
  cat(paste0(">id.", i, "\n"))
  cat(generate_protein(len = i), "\n")
}
```

```
>id.6
```

```
WAFNNK
>id.7
EPWVNMW
>id.8
QCSASKFN
>id.9
VGVWGSWIR
>id.10
ADDDPLQIKV
>id.11
MMDQWVNSTDR
>id.12
WVDSCDHNKPWQ
>id.13
HQNNIFMPIHAAA
```

**Section 5) BLASTp search against nr — are your peptides "unique in nature"?**

Question 5. Take your FASTA-formatted peptides from Q4, run them as a single BLASTp search against the Non-redundant protein sequences (nr) database, and fill in the table:

| Length (aa) | Best hit % identity | Best hit % coverage | Unique? (Y/N) |
|---|---|---|---|
| 6 | 100% | 100% | N |
| 7 | 100% | 100% | N |
| 8 | 87.50% | 100% | Y |
| 9 | 100% | 89% | Y |
| 10 | 100% | 90% | Y |
| 11 | 81.82% | 100% | Y |
| 12 | 75% | 100% | Y |
| 13 | 100% | 62% | Y |

Question 5a. At which sequence length do your randomly generated peptides start to look "unique in nature" (i.e. no 100% coverage + 100% identity hit)?

Response: Based on the table, random peptides start to look "unique in nature" at 8 amino acids, since length 6 and 7 still have 100% identity and 100% coverage hits, while length 8 no longer does.

Question 5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe.

Response: Very short random peptides are almost always found in nr because the sequence space for small L is relatively limited compared with the enormous number of known protein fragments already represented in nature and in databases. As peptide length increases, the number of possible sequences grows exponentially as 20^L, so it quickly becomes far larger than the known protein universe, making an exact full-length match much less likely.

**Section 6) Connecting your findings to immunology (MHC class II and T-cell activation)**

Question 6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

Response: The minimum peptide length is likely around 8 amino acids or a bit longer, because that is where peptides begin to look less common and more distinctive according to the results from Q5. Presenting very short peptides would be a bad design choice because they are too likely to occur by chance in many unrelated proteins, including self proteins. Thus, they would not reliably distinguish self from non-self. Longer peptides sample a much larger sequence space and are therefore more likely to be specific to a particular pathogen-derived protein. This is what makes immune recognition more selective and reduces the chance of false matches.